Group X Progress Report:
My Group's Project Name

Omar Alam, Sathurshan Arulmohan, Kalp Shah, Jay Sharma
{alamo2,arulmohs,shahk50,sharmj28}@mcmaster.ca

1 Introduction

Paraphrase detection is the task of determining whether two pieces of text express the same meaning, even if they use different words or sentence structures. This problem is useful in applications such as plagiarism detection, semantic search, duplicate question detection, and large-scale text processing. The task is challenging because surface-level similarity is often misleading: two sentences can share many words but mean different things, while two sentences with very different wording can still be semantically equivalent. As a result, an effective system must go beyond keyword matching and capture the underlying meaning of each sentence in context.

In this project, we study paraphrase detection as a regression problem. Given two tokenized English sentences, the model predicts a score between 0 and 1 that represents how strongly the sentences paraphrase each other. A low score indicates little semantic overlap, while a high score indicates that the two sentences communicate nearly the same idea. To support this task, we use a subset of the ParaNMT-50M dataset, which provides large numbers of sentence pairs with similarity scores. Our overall goal is to build and evaluate models that can learn both word-level similarity and deeper semantic relationships, allowing the system to assign reliable paraphrase scores to unseen sentence pairs.

2 Related Work

Past work on paraphrase detection usually looks at the problem in two ways: either deciding if two sentences are paraphrases or giving them a similarity score. One of the best-known datasets in this area is the Microsoft Research Paraphrase Corpus [1]. A closely related area is semantic textual similarity (STS), where the goal is to predict how similar two sentences are on a scale instead of making a yes-or-no decision [2]. This is especially relevant to our project because we also treat the problem as score prediction. These works show that paraphrase detection is not just about matching words. Two sentences may use different wording and still mean the same thing.

Another important area of related work is sentence embeddings. Back-translated text has been used to create large paraphrase datasets for learning sentence representations [3]. ParaNMT-50M, which we use in this project, builds on that idea [4]. More recent methods use transformer models to create better sentence embeddings. Sentence-BERT made it easier to compare whole sentences using cosine similarity [5], and SimCSE further improved sentence representation quality using contrastive learning [6]. No previous paper matches our exact project setup, but these papers are the closest to what we are doing because they focus on paraphrase datasets, similarity scoring, and sentence-level meaning.

3 Dataset

We use the ParaNMT-50M dataset for this project. Rather than creating new labels manually, we use the similarity score provided with each sentence pair as the supervision signal for training and evaluation. Each example contains two English sentences and a real-valued score between 0 and 1. Higher scores indicate that the two sentences are more semantically similar, while lower scores indicate weaker similarity. Since our task is formulated as a regression problem, this score is treated as the target value the model must predict.

Each row in the dataset contains three fields: sentence 1, sentence 2, and a score. This format directly matches the paraphrase scoring task because the model receives a pair of sentences as input and learns to predict a continuous similarity label. For example, sentence pairs with almost identical meaning receive scores close to 1.0, while

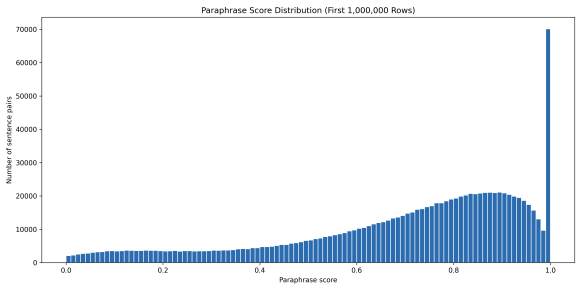

Figure 1: Distribution of paraphrase scores for the first 1,000,000 rows of ParaNMT-50M.

pairs with weaker semantic overlap receive lower scores.

To better understand the labels, we plotted a histogram using the first 1,000,000 rows of the dataset. The x-axis represents paraphrase scores from 0 to 1 using bins of width 0.01, and the y-axis shows the number of sentence pairs in each bin. As shown in Figure 1, the distribution is not uniform. Instead, the dataset is concentrated more heavily toward higher scores, and the largest spike occurs at 1.0. This suggests that highly similar sentence pairs appear much more frequently than weakly related ones in the sampled data.

Because the labels are continuous rather than discrete classes, traditional class balance does not apply directly. However, the score distribution is still clearly imbalanced across the regression range. In particular, the upper score bins contain substantially more examples than many of the lower score bins. This means the dataset is skewed toward highly similar sentence pairs, which may affect model behavior during training and evaluation. For this reason, understanding the label distribution is important when interpreting performance and analyzing prediction errors.

4 Features and Inputs

For the paraphrase detection task, the input method can drastically change the performance of any implemented model. Ideally, sentence pairs that are more likely to be paraphrased should be represented closer in the feature / input space, compared to sentence pairs that have a low similarity score. This leads to the problem of tokenization and deciding on the input space, which has a vast library of solutions. Bag of Words representations of each word in a sentence would be somewhat unreasonable as the vocabulary size is extremely high for our dataset with 50 million sentence pairs. The resulting feature vectors would also be extremely sparse which would significantly increase the difficulty of learning, not to mention the large amount of computation required to process a sentence. For our project, we decided to use an embedding model which embeds individual words into a reasonably low dimension space considering available compute.

The FastText 300 dimensional word embedding model trained by Facebook Research on the Common Crawl dataset **with subword information**. The subword information allows the embedding to effectively tokenize words that are rare or out of vocabulary by breaking them down into smaller units for which the corresponding embedding vectors get summed up to represent the full word. An example may be:

<Impairment> may be broken down using a 3-gram approach to yield <Im, Imp, mpa, pai, air, irm, rme, ent, nt>. Embeddings for each of these subwords can be added up to create a representation for the full word token.

Using this approach helps us avoid out of vocabulary errors when using embeddings. With the FastText embedding model, each sentence can be represented as a 2D 300 dimension column wise matrix of all the word tokens in it. The simple_preprocess tokenizer is utilized from the **gensim** library to tokenize sentences into words. A sentence embedding will then have the following format:

$$S = \begin{pmatrix} e_{1,1} & e_{1,2} & \cdots & e_{1,n} \\ e_{2,1} & e_{2,2} & \cdots & e_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ e_{300,1} & e_{300,2} & \cdots & e_{300,n} \end{pmatrix} \in \mathbb{R}^{300 \times n} \tag{1}$$

where each column $e_{:,j} \in \mathbb{R}^{300}$ is the FastText embedding of the j-th word token, and n is the number of tokens in the sentence.

5 Implementation

There are multiple architecture types that can be used for detecting the paraphrasing probability between two sentences. Primarily, the architecture type needs to be well suited to processing text to extract meaningful abstract features for further processing by a fully connected network. Several architecture types were considered:

1. Transformer: Arguably the most performant architecture for this task as it excels at natural language processing tasks and has little issues with capturing long term dependencies since it uses attention mechanisms.

2. Recurrent Neural Networks (RNNs): Recurrent neural networks are also a good choice for this task since they are able to track somewhat long dependencies as they have a persistent hidden state between each token evaluation.

3. 1D Convolutional Neural Networks (CNNs): CNNs are traditionally used for image processing, but may also be used for NLP in their one dimensional form. For this task specifically, it may be possible to use a 2D convolutional network. In general, these are not great at capturing long term dependencies since convolutions work on localized sections of data.

One of the submissions for the Codabench competitions was using a transformer which was fine tuned on the provided dataset. The fine tuned BeRT transformer had an amazing accuracy of 99.17 calculated using the accuracy metric $(1 - \text{MSE}) \cdot 100$. Since the project already had a transformer result from this submission, we decided to implement the RNN architecture for result diversity.

There are many variants of RNN architectures such as encoder-decoder pairs, gated recurrent units (GRU), and long short-term memory (LSTM) networks to name a few. We decided to proceed with the LSTM network architecture for this task since it has more parameters which may allow it to capture more meaningful information about the task from the dataset compared to GRUs or other types which generally consist of fewer training parameters. LSTMs aim to solve the long term dependency problem by implementing a hidden state, context vector, and 3 gates for input, forget, and output. The individual gates take in the next token and decide what to forget from the context and hidden state vectors which controls what gets "forgotten" and "remembered" for when the next token is processed. An architecture diagram for LSTM is shown in Figure 2.

With the embedding dimension being 300 for each token, the input size is (300×1) with the hidden state size being selected as (200×1) for compression and faster training. For the model implemented, we use bi-directional LSTMs with

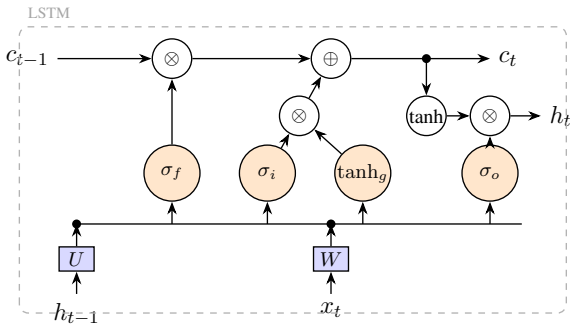

Figure 2: LSTM cell. c_t: cell state, h_t: hidden state, x_t: input token. Circles: σ (sigmoid), tanh. Operators: $\otimes$ (pointwise multiply), $\oplus$ (pointwise add). Gates f, i, g, o are the forget, input, candidate, and output gates.

the first LSTM processing tokens in the sentence from left to right and the second LSTM processing tokens from right to left in the same manner as a bi-directional RNN. A diagram of the bi-directional LSTM is shown in Figure 3.

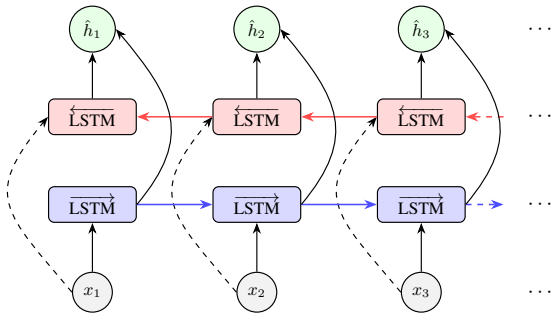

Figure 3: Bi-directional LSTM architecture. The forward LSTM (blue) processes tokens left-to-right producing $\overrightarrow{h}_t$; the backward LSTM (red) processes right-to-left producing $\overleftarrow{h}_t$. Both receive the same input x_t (dashed arrows). The output $\hat{h}_t = [\,\overrightarrow{h}_t \,\|\, \overleftarrow{h}_t\,]$ concatenates both hidden states.

Each sentence from the pair are passed through the bi-directional LSTM and their hidden states get concatenated to form an (800×1) vector that is passed to a fully connected network with 800, 400, 200, 100 and 1 inputs per layer with the final layer containing the paraphrasing score.

The model was initialized to random parameter values with the following training procedure:

- Huber Loss Function (δ is a hyper-param) :

$$L_\delta(a) = \begin{cases} \frac{1}{2}a^2 & |a| \le \delta \\ \delta \cdot (|a| - \frac{1}{2}\delta) & \text{else.} \end{cases}$$

- Adam Optimizer with learning rate $\alpha = 0.004$ for 20 epochs.

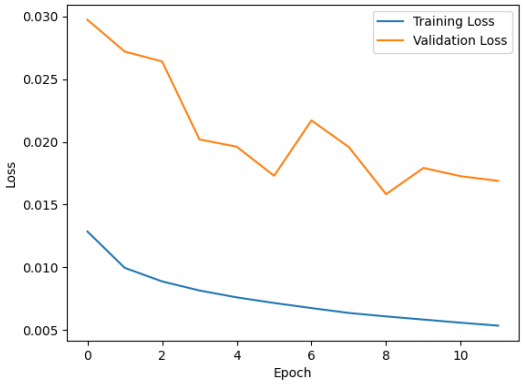

Figure 4: Training and Validation loss vs Training Epoch for Bi-LSTM model.

- Dataset used: 1,000,000 subset from the 50,000,000 full dataset with the following split:
  - Training set: 70% (700,000) samples.
  - Validation set: 10% (100,000) samples.
  - Training set: 20% (200,000) samples.
- Dropout between FCN layers with probability 0.2.

The small subset was taken due to compute constraints. Training yielded the training and validation loss curves shown in Figure 4. When comparing on Codabench, this model outperforms all of our baselines (majority and random) achieving 93.6.

6 Evaluation

The dataset was split into training, validation, and testing sets comprising 70% (700,000 samples), 10% (100,000 samples), and 20% (200,000 samples), respectively. The original score distribution, shown in Figure 1, is skewed toward higher scores for the first 1,000,000 datapoints of ParaMt-50M.

To ensure a fair evaluation, a balanced dataset was constructed. Specifically, the first 10 million datapoints from ParaMt-50M were sampled, and 100,000 samples were selected from each bin of width 0.01. This process resulted in a more uniform distribution across score ranges, as illustrated in Figure 5. The same 70/10/20 split for training, validation, and testing was maintained on this balanced dataset.

Model performance was evaluated using a custom accuracy metric based on mean squared error (MSE), defined as:

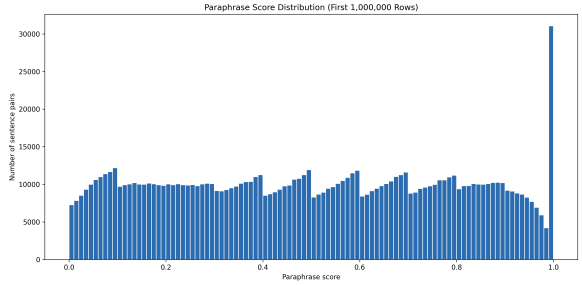

Figure 5: Distribution of balanced paraphrase scores used for evaluation.

Method	Category	Score (%)
Majority baseline (mean label)	Simple baseline	≈ 91.6
Bi-LSTM + FastText + FC head (ours)	Our approach	93.6
Fine-tuned RoBERTa (another group)	Classmate method	99.17

Table 1: Comparison of paraphrase scoring methods on the shared evaluation setup. Scores use $(1-\mathrm{MSE})\times 100$ as in Section 6. The majority baseline figure reflects evaluation on the balanced split of ParaNMT-style data; neural models were evaluated under the same metric on Codabench.

$Score = (1.0 - MSE(y_{true}, y_{prediction})) * 100.0$

A perfect prediction (MSE = 0) yields a score of 100%.

Additionally, the Humber loss function was used during training to monitor model performance over time.

7 Comparison to Others

A central question for this project is how our approach compares not only to simple baselines but also to other groups working on the same task under similar constraints. Table 1 summarizes results using the same Codabench-style metric defined in Section 6, namely Score $= (1 - \mathrm{MSE}(y_{\mathrm{true}}, y_{\mathrm{pred}})) \times 100$, so that higher values correspond to lower mean squared error between predicted and gold similarity scores.

As described in Section 5, our system encodes each sentence with FastText subword embeddings, passes each sentence through a bi-directional LSTM, concatenates the final hidden representations for the pair, and regresses a scalar score through a small fully connected stack trained with Huber loss. This architecture is deliberately lighter than a transformer: it was chosen partly to diversify the set of solutions in the course and to stay within

our compute budget, while still learning non-trivial functions of the input rather than predicting a single global mean.

A different team in the course fine-tuned a RoBERTa model (a pretrained transformer encoder) on the same paraphrase regression task and submitted their model to Codabench. RoBERTa builds on the BERT family but uses improved pretraining recipes and remains a strong general-purpose encoder for sentence-level NLP. In their setup, the model can attend over all token positions in both sentences jointly (depending on how the pair is formatted), and it benefits from large-scale language modeling on text corpora before any task-specific fine-tuning. That combination typically yields rich contextual representations that go well beyond bag-of-embeddings or fixed-window recurrence.

We also include the majority baseline from Section 6, which always predicts the mean training label. It does not read the sentences at all; it merely minimizes squared error among *constant* predictors. Its score in the high 80s–low 90s under $(1 - \mathrm{MSE}) \times 100$ is therefore not analogous to 'accuracy' in classification: as discussed when interpreting regression metrics, even a trivial mean predictor can achieve a seemingly high number because the metric is bounded and rewards any reduction in average squared deviation from the global mean.

So are these results what we expected? Broadly, yes. The ordering in Table 1 matches standard expectations for modern NLP: a strong pretrained transformer with task-specific fine-tuning (the classmate RoBERTa system) achieves the strongest score, our recurrent model sits clearly above the non-input baseline but below the transformer, and the majority baseline trails the neural methods despite its deceptively high raw percentage. The gap between our Bi-LSTM (93.6%) and RoBERTa (99.17%) is large in absolute percentage points but is consistent with known differences in representational capacity, pretraining, and the ability of self-attention to model long-range and subtle semantic contrasts that are hard for a finite-width LSTM to capture from word embeddings alone. The smaller gap between the majority baseline and our Bi-LSTM reflects that moving from a constant predictor to a model that actually conditions on the sentence pair yields real gains on the regression task, even though the metric's scale makes all numbers look superficially high.

In the end, we did not expect our model to match a state-of-the-art fine-tuned transformer; the main takeaway for us is that there remains substantial headroom on this benchmark, and that architecture choice (recurrent vs. transformer) and pretraining matter greatly for paraphrase scoring, which is in line with the related work surveyed in Section 2.

8 Error Analysis

Initially, when training on the skewed dataset shown in Figure 1, the model performed worse than the majority baseline. This is expected, as the dataset contains significantly more datapoints with scores close to 1 and relatively few with scores near 0. Consequently, the model failed to learn effectively across the full range of score variations.

However, after updating the dataset to a more uniform distribution, as shown in Figure 5, model performance improved substantially. The updated model outperformed both the majority baseline and the previous model trained on the skewed dataset, achieving an improvement of 8%.

Furthermore, the model performs worse when evaluating longer text pairs compared to shorter ones. This is because longer texts contain more semantic content and complexity, requiring the model to capture and compare multiple semantical meaning. As a result, generating a single similarity score becomes more challenging, leading to reduced performance on longer sequences.

9 Conclusions

In this project, paraphrase detection was studied as a regression task, where the goal was to predict a continuous similarity score between pairs of sentences. Using a subset of the ParaNMT-50M dataset, a bi-directional LSTM model combined with FastText embeddings was implemented to capture both word-level and contextual semantic information. The model successfully learned meaningful sentence representations and achieved strong performance, outperforming baseline approaches.

A key finding of this work is the significant impact of dataset distribution on model performance. Training on the original skewed dataset led to poor generalization, particularly across lower similarity scores. By constructing a balanced dataset, the model was able to learn more effectively across the full score range. This highlights the importance of considering label distribution, even in regression settings.

Overall, the results demonstrate that the proposed approach is effective for paraphrase scoring under constrained computational resources. However, there are several opportunities for future improvement, including leveraging transformer-based architectures, incorporating sentence-level embedding models, and exploring more robust methods for handling long sequences and semantic composition.

Team Contributions

Omar Alam worked on training on the model and writing the Features and Inputs, and Implementation sections. Sathurshan Arulmohan worked on writing Evaluation, Error Analysis, Conclusion sections and project managing. Jay Sharma worked on dataset split and writing the Introduction, Related Work and Datset sections. Kalp Shah worked on writing Comparison to Others sections and managing Codabench by helping others and debugging issues that arises.

References

[1] W. B. Dolan and C. Brockett, "Automatically Constructing a Corpus of Sentential Paraphrases," in *Proc. Third Int. Workshop on Paraphrasing (IWP2005)*, 2005.

[2] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "SemEval-2017 Task 1: Semantic Textual Similarity Multilingual and Crosslingual Focused Evaluation," in *Proc. 11th Int. Workshop on Semantic Evaluation (SemEval-2017)*, 2017.

[3] J. Wieting, J. Mallinson, and K. Gimpel, "Learning Paraphrastic Sentence Embeddings from Back-Translated Bitext," in *Proc. 2017 Conf. on Empirical Methods in Natural Language Processing*, 2017.

[4] J. Wieting and K. Gimpel, "ParaNMT-50M: Pushing the Limits of Paraphrastic Sentence Embeddings with Millions of Machine Translations," in *Proc. 56th Annual Meeting of the Association for Computational Linguistics*, 2018.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing and the 9th Int. Joint Conf. on Natural Language Processing*, 2019.

[6] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple Contrastive Learning of Sentence Embeddings," in *Proc. 2021 Conf. on Empirical Methods in Natural Language Processing*, 2021.